Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX

Validate the statistical result first. Then accelerate the bottleneck.

Wenxin Jiang · Jian Yin
Department of Biostatistics, City University of Hong Kong · PyCon US 2026

github.com/LucaJiang/FastStatisticalModels4Python

1. Why simulation is the statistical test harness

Real biomedical data rarely comes with ground truth. Simulation lets us know what should be recovered, calibrated, or preserved.

Define target what should be recovered, calibrated, or estimated

Generate scenarios vary signal, dimension, sample size, and hard cases

Reference first write the readable implementation before optimizing

Validate equivalence, recovery, null calibration, tolerance

Scale only then measure bottlenecks and choose an engine

Governing rule
Same question. Same result. Then faster code.

2. What a statistician has to decide

Question	Decision before timing
Scientific target	estimator, statistic, null, stopping rule
Ground truth	labels, known effect, or nominal type-I rate
Acceptance criteria	exact match, tolerance, calibration, recovery
Failure meaning	implementation, method, or systems limit
Scale stress	dimensions that make work expensive

Reference
golden oracle

Scenario grid
property / fuzz / load tests

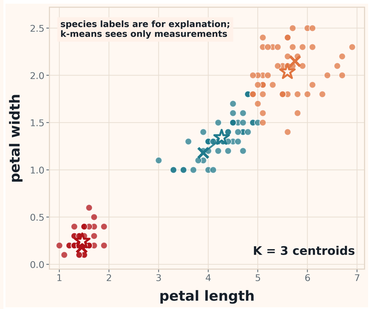
Gate
optimization is admitted only after validation

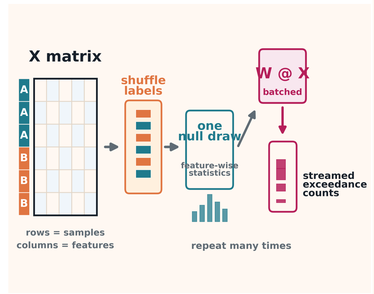
3. Two workloads, two pressure shapes

These workloads stand for two common pressure shapes in statistical simulations.

k-means
Iterative fitting: each update depends on the previous state.

permutation test
Resampling inference: repeated null draws across many features.





Validation checks
k-means: same data, initialization, stopping rule; compare inertia and recovery.
permutation: same permutation stream and p-value definition; check calibration before runtime.

4. Evidence examples, scoped

Claim type	Committed evidence used in the talk
k-means equivalence	max relative inertia difference 3.1×10^{-14}; tolerance 10^{-8}
permutation equivalence	max p-value diff 0.0; max statistic diff 9.4×10^{-16}
null behavior	MacBook null calibration mean $p \leq 0.05$ was 0.051
A100 boundary	first streamed-reduction win at $n = 5{,}000$, $p = 10{,}000$, $R = 5{,}000$, batch $R = 8{,}192$
A100 scope	largest slide-level measured speedup 8.54x at $p = 500{,}000$, $R = 5{,}000$

Timing semantics
Speedup = matched CPU matrix baseline / A100 streamed full end-to-end.
Compile excluded, transfer included, kernel-only excluded.

5. Choose the smallest tool that matches the bottleneck

Simulation signal	Conservative tool choice
target not recovered	fix statistic / model first
hot scalar CPU loop	Numba
dense distance algebra	NumPy / BLAS
shared-array repeats	threads / free-threaded context
large device-resident batch	JAX / A100
giant temporary	rewrite or stream
non-monotone sweep	tune under constraints

trust first **compile loops** **use algebra** **batch for device**

6. What to report with every speed claim

- validation target and acceptance criteria;
- tier: local validation, server CPU, or A100;
- cold/warm timing and compile treatment;
- transfer, allocation, and collection treatment;
- unavailable/OOM cells as unavailable, not as CPU wins;
- the smallest tool that preserves the statistical result.

Takeaway

1 Simulate known behavior

2 Preserve the statistic

3 Measure the bottleneck

4 Choose the smallest tool

Clear enough to trust. Fast enough to scale.

github.com/LucaJiang/FastStatisticalModels4Python